Design document — Kipu Pay corpus anonymization

Reto Frontier "Vende los datos sin vender a la gente" · Pipeline built with the Pi coding agent, served as deterministic code behind an HTTP service

Summary. Kipu Pay (Colombian fintech) wants to sell 3 years of support chats, internal docs and customer tables to a vendor training customer-service models. The deal closes only if **the data stays useful for training** and **no real person is exposed**. We build a **rules-first pipeline** producing a 1:1 copy (2,028 chats, 26 docs, 6,243 rows) where each person gets **one consistent, realistic pseudonym across all sources**, quasi-identifiers are **generalized, not deleted**, and business content is kept. Scored on **Leaks 45 · Utility 35 · Consistency 20**. Service regenerates output in **<2 h** and answers `/anonymize` in **<60 s / 50 records**. Honest target: **~85–88 rules-only**; optional Jev step for extra recall.

1 · Understanding the problem

What counts as PII

Type	Examples → treatment
Direct	names (full, short, CAPS no accents, filename slug), cédula ± dots, passport, email incl. "arroba/punto", phone ± +57, address, birth date, plate, `<@UID>` → same-category pseudonym
Indirect	neighborhood + occupation + year that single someone out → occupation→sector, neighborhood→another in same city, year→"hace unos años", in table **and** text
Hidden	PDF metadata, front matter, HTML/XML comments, XML attributes, attachment & doc filenames → same engine on every field

Kept on purpose: orgs/banks/merchants/regulators (Bancolombia, Nequi, Superfinanciera), places, public figures, Kipu Pay/Kipi, internal IDs (CL-, TX-, KP-, PQR-) so tables still join, amounts, business dates, city — not personal on their own; removing them costs utility.

Constraints: run <2 h · anonymize <60 s · no PII products/libraries · 600 req/min · sources grow mid-event (old people keep pseudonyms) · 3 runs · online 19:00→demo day. **Assumptions:** `cliente_id`/KP-`id` = identity; chat users link by email/name; namesake tie-breakers live in another source; new data never edits old; PDFs have text layer; data is mostly Colombian.

Hard cases we handle

Case	Handling
Namesakes (2× Camila Arango Velásquez)	identity = `cliente_id`, never the name
Agent named like a client (Luis Santander)	separate employee/client registries; role from source
Surname = place (Santander)	replaced only inside a matched person name
Short forms ("Hola Manuela", `reporta_a="David Mejía"`)	variant → matching form of pseudonym
CAPS / no accents; spelled emails	normalized matching; dedicated regex, style kept
Bot/agent repeats client data; third parties in notes	detect on every text, not by author; registry then name list
Quasi-IDs split across chat + CSV	same generalization in both
Data growth; live unseen records; cold start	HMAC + saved map; shared core; works with regex+name list before any run

Known gaps (honest) & why

Gap	Reason / mitigation
Rare/foreign names of unknown third parties	no trained NER (banned libs, 1 day) → leak review + optional Jev
Non-Colombian ID formats	a rule per format; generic digit/email rules instead; people always pseudonymized
Namesakes in free text with no linking ID	coreference is research-grade → default to person active in thread; merges possible
Nicknames ("Mafe"), pronoun references	needs curated dictionary/coreference → only Slack display names
Other quasi-IDs (rare condition + city, unique titles)	full k-anonymity needs time, over-generalizes → only the defined combo
Ages/birthdays in words; OCR/images	unreliable with rules; out of scope
Scale to millions of records	in-memory, single process → would need DB map + workers

Out of scope: LLM rewriting (hurts similarity, can alter data) · LLM over every record (slow, costly, non-deterministic) · trained NER · rebuilding PDFs · auth/scaling/UI.

2 · Architecture & trade-offs

Ingest → Normalize → Person registry → Detect → (Jev classify, optional) → Assign pseudonyms → Replace & generalize → Write & check → HTTP API, with a persisted pseudonym map shared by `/run` (background job, 202 + job_id) and `/anonymize` (sync, in-memory). Details in the architecture document.

Decision	Rejected	Why
Deterministic code at request time	runtime agent	60 s limit, exact consistency, dead service burns a run; Pi builds it
Rules-first detection	LLM-first	rules ≈ 88 benchmark, 0 tokens, deterministic
Registry from tables first	identity by chat author	stable IDs solve namesakes; author-only misses third parties
HMAC(salt, key) + saved map	random / plain hash	stable across growth; hashes reversible, no category
Same-category pseudonyms	[REDACTED], John Doe	utility + structure points (redaction = 56)
In-place span replacement	model rewrite	keeps text & business data
Generalize quasi-IDs in table + text	delete / chats only	indirect leak judged on union
One core for /run + /anonymize	separate logic	same pseudonym batch & live
Jev only for typed decisions on leftovers	generative LLM	cheap, fast, cached; pipeline works without it
FastAPI, always-on container + volume	serverless / free tier	long job, up for hours, map survives restarts

Failures: detection miss → leak sweep flags record · Jev down → timeout, rules fallback · rate limit → throttle + retry, job reports `failed` · restart → reload map & last output · bad input → 422, never crash.

3 · Plan to measure

Test	Expected
Validator; 1:1 vs record_index (8,297)	OK; 0 missing / 0 extra
Cédula `32.355.343` (pub_02)	gone everywhere, not as hash
Attachment `camila_restrepo` (pub_01)	slug replaced
`Nequi` (pub_03)	kept unchanged
Namesakes CL-12538 (pub_04)	two pseudonyms
Ana Lucía Marulanda all forms (pub_05)	one pseudonym everywhere
13 sample tests (memo author, front matter, quasi-ID, filename, bank, regulator…)	13/13
`reporta_a`; spelled email; PDF metadata doc_09	consistent short form; replaced; keys kept
Live: known / new / namesake person; cold start	same / new unique / two; <60 s
Two runs; subset→full growth simulation	identical pseudonyms

Metric	Target
Originals found by leak sweep	0
Public + sample tests	18/18
Coverage · uniqueness · stability	100% · 100% · 100%
/run time · /anonymize (50)	<10 min · <5 s
Tokens per run	0 rules-only; Jev calls logged

Process: sample vs ideal output → local scorer (18 tests) + leak sweep on full data → test deployed URL end to end → **spend run #1 before sources grow** (30%) → fix → runs #2/#3 after growth (70%). **Cost:** 0 tokens rules-only, ~2–5 min per run.